**CMSC436 Project 4 – Assigned Tuesday March 3, 2026**
**Due Thursday March 26, 11:59PM**

0 – You can do this app/project with another CMSC436 student (you do not have to). One submission only for 2 students if you choose to do it as a pair. Include both names in the top comments of your code (inside MainActivity) and submit as a group/pair. The project must be called project4 and the package must be com.example.project4. The project will be tested on emulator **Pixel 5 API 34**.

1 - Create an Android App using the Empty Views Activity template

2 – The app works in vertical orientation only (but you do not need to submit the AndroidManifest.xml file). This app has only 1 screen. This app is a simple version of the **game "Galaxian"**. It is ok to leave the status bar (you can also get rid of it if you want).

3 – When the app starts, the user is ready to play the game. There is one row of enemies at the top of the screen (6 are shown on the screenshot, but there could be anywhere from 5 to 10). The number of enemies should be a constant in your Galaxian class and you should use that constant wherever it is needed (example: by changing that constant from 6 to 8, we would now have 8 enemies). At the start, all the enemies should be on the same horizontal line and the spacing between them should be constant, they should look clean. You can hard code the number of pixels left of the first enemy and right of the last enemy).

There is a ship somewhere at the bottom of the screen. At the tip of the ship (the "tip" location does not have to be perfect, but the bullet should be centered), there is a bullet, ready to be fired. The game starts immediately. The enemies come down, in random order, and with random timing (not necessarily at some frequency; using a frequency for that is ok though). They come down at a 45 degrees angle (either left or right, you choose) and will bounce off the wall at a 45 degrees angle (always) as they keep coming down. They should be either right at the wall or reasonably close to the wall when they bounce.
If an enemy goes through the bottom of the screen, that enemy should start again at the top of the screen.

**You should have 2 transparent pngs: one for an enemy, one for the ship; the files should be in the drawable directory and named enemy.png and ship.png.** It is ok to use AI to generate the pngs.

4 – The user can move the ship from left to right and right to left (always horizontally) by moving the finger (note that the finger **does not** have to be on the ship for the ship to move).

5 – If an enemy hits the ship, the ship is destroyed and no longer visible. That enemy is also destroyed and no longer visible. The bullet is no longer visible either and no enemy can be destroyed by the bullet. At that point, you should display "YOU LOST" (not in tiny characters) somewhere in the middle of screen (you can pick the location; the text does not have to be centered). It is fine if the enemies are still coming down (you can also stop the enemies from moving, or even not show them at that point). A Player has only one life in this game. At this point, you should also draw somewhere on the screen and report the number of enemies hit and the best score and whether this is or not the new best score. Note that the absolute best score cannot go over the number of enemies when we started the game.

6 – Note that two enemies can "collide" with absolutely no effect; they keep going as nothing had happened.

7 – The user can shoot a bullet, only one at a time, but tapping once on the screen. The bullet should reload automatically (at the tip/just above of the ship) when it goes off screen or when it hits an enemy. The bullet moves vertically towards the top of the screen. If a bullet hits an enemy, that enemy is destroyed and is no longer visible. The goals is to destroy all the enemies. If the user destroys all the enemies, then the user won and you should display "YOU WON" (not in tiny characters) somewhere in the middle of screen (you can pick the location; the text does not have to be centered). Like in #5 above, also display the game statistics.
Note: if it happens that a single bullet destroys 2 enemies, that is fine.

8– Every time the user shoots, it makes a sound. **You should name your sound file fire.wav and place it in the raw directory** (so that the TAs do not have to copy and paste your file). You need to create the raw directory.

9 – You should keep track of how many enemies the user destroys for the current game. This is your score. The app stores (persistent data, in shared preferences) the user's best score (the first time we use the app, the best score is 0).

10 - Your app should include a Model (class Galaxian in file Galaxian.kt) that encapsulates the functionality of the game. Obviously, the model needs to be used (and used in a way that makes sense) within the app. Some things that should be included in the Model: the ship and appropriate attributes, bullet position, speed, size, enemies including position, speed, …, methods to move the ship, move the bullet, detect if the

bullet has hit an enemy, if an enemy has hit a wall, if an enemy is colliding with the ship, .. If you think you need to add more classes for the Model, then you could have up to 3 more classes: Ship, Enemy, and Bullet (and you must name them that way in files Ship.kt, Enemy.kt, Bullet.kt). It is your choice to have or not these classes.

**Submission Details:** on Gradescope, (at least) 4 Kotlin files: MainActivity.kt, Galaxian.kt (the Model), GameView.kt and GameTimerTask.kt (and more files if appropriate – see #10 above)

**Grading Details:**

| Item | Points |
|---|---|
| Enemies show at the start of the app | 10 |
| Bullet and ship show at the start of the app | 5 |
| Game starts | 5 |
| Ship moves correctly | 5 |
| Bullet can be fired and moves correctly | 5 |
| Enemies bounce off the wall correctly | 5 |
| Enemies are destroyed correctly (and no longer visible) | 10 |
| Ship is destroyed correctly (and no longer visible) | 10 |
| If the ship is destroyed, the bullet is no longer visible and enemies cannot be destroyed by the bullet | 5 |
| A sound is played when the bullet is fired | 5 |
| When the use wins or loses, (correct) text shows | 10 |
| Bullet reloads correctly | 5 |
| Model (Galaxian class) is coded and is correct | 10 |
| Model (Galaxian class) is used correctly | 10 |
| **Total** | **100** |

**Grading notes:**

1 – In canvas, zipping the files will result in a 10 points penalty.
2 – Not naming the files as specified above will result in a 10 points penalty.

**Suggested Plan:**

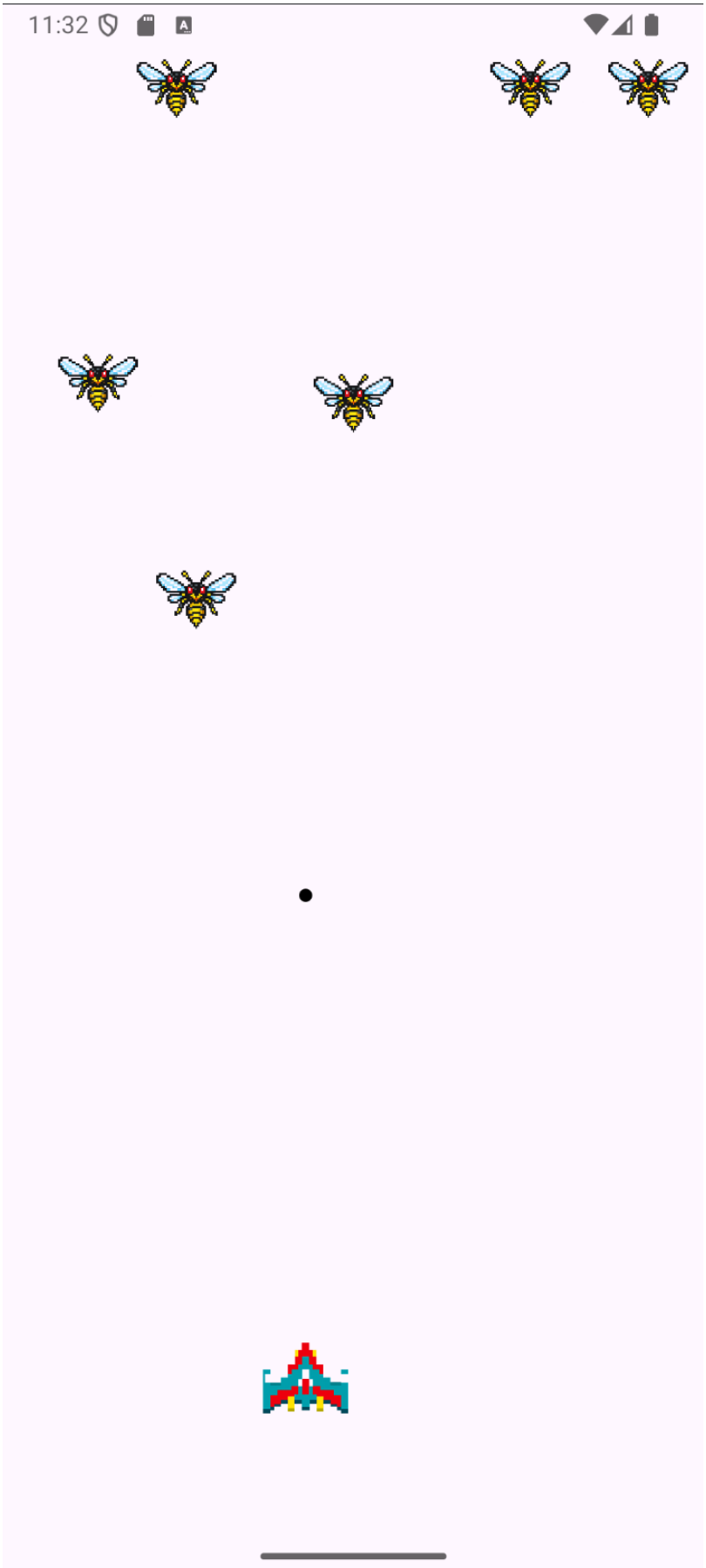
1 - Show the ship somewhere at the bottom of the screen
2 - Show the bullet at the tip of the ship
3 – Show the enemies at the top of the screen
4 – Enable the ship to move when the user moves the finger
5 – Enable the ship to fire a bullet

6 – Enable the bullet to be fired
7 – Start the game (enemies start coming down in random order, at some random frequency)
8 – Make the enemies change direction when they hit the wall
9 – Destroy an enemy that has been hit by the bullet
10 – Reload the bullet appropriately
11 – Keep track of how many enemies have been destroyed
12 – When the ship is hit, destroy the ship and display the "YOU LOST" message and the statistics (number of enemies destroyed)
13 – If the user wins, display the "YOU WON" message and the statistics

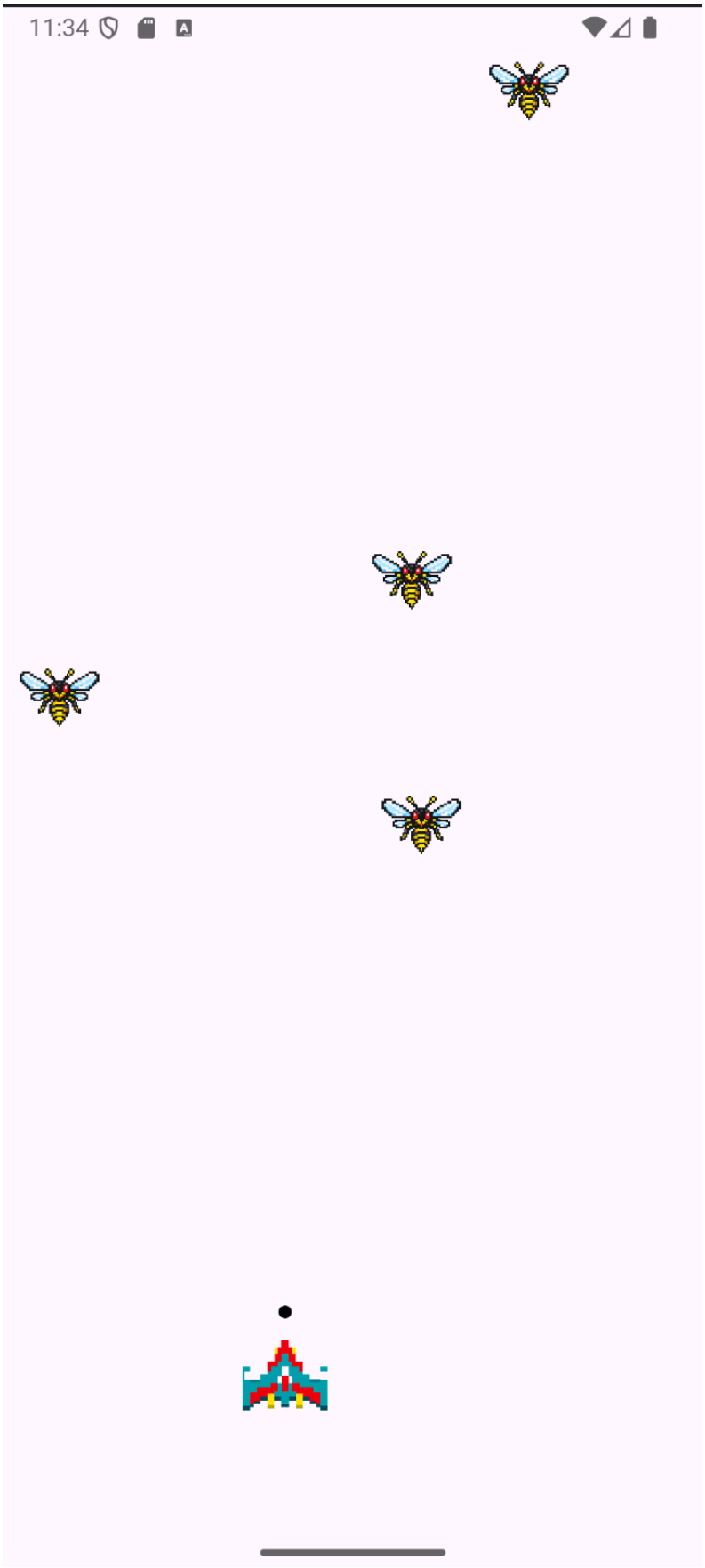
**<u>The start of the app (everything is showing, some enemy will start coming down soon).</u>**

**The game started, enemies are starting to move down**

## Enemies are coming down, the player fired a bullet



**The user has already destroyed 2 enemies**



**The user destroyed all the enemies and has won**

**The user destroyed 4 enemies but lost**